e-Yantra Robotics Competition 2025-26

Technical Project Report

Theme: Crop Drop Bot

Institution: Zeal College of Engineering and Research

Team Members

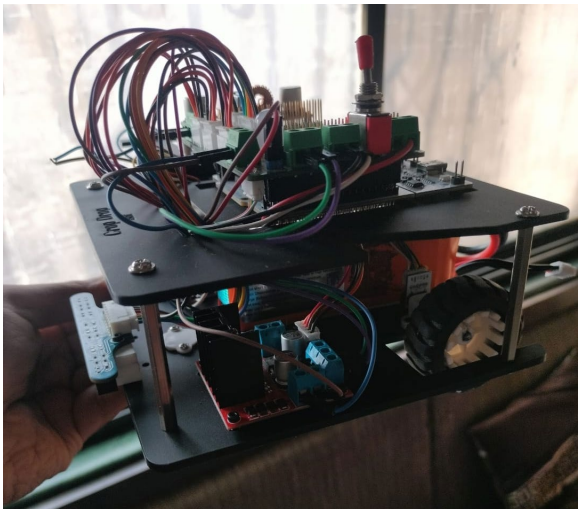
Mohit (Team Lead)

Sanyog Dahotre

Aaryan Nikam

Registration, Introduction & Theme Selection

1.1 Introduction



The e-Yantra Robotics Competition (eYRC) is a flagship initiative by IIT Bombay. It provides a unique platform for engineering students to implement theory into practice through a project-based learning approach. This report documents the technical journey of the team from Zeal College of Engineering and Research in the 2025-26 edition of eYRC.

1.2 Team Formation

Our team was formed with a shared passion for robotics and automation. Led by Mohit, who took on the responsibility of the College Group Team Lead, the team comprises Sanyog Dahotre and Aaryan Nikam. Each member brought a distinct skill set to the table, ranging from embedded systems and hardware assembly to algorithmic problem solving and simulation.

1.3 Theme Selection: Crop Drop Bot

Among the various themes offered in eYRC 2025-26, our team unanimously selected the **"Crop Drop Bot"** theme. This theme focuses on autonomous navigation and service robotics in an agricultural context. The challenge involves designing a robot capable of traversing a greenhouse environment, identifying service zones, and performing pick-and-place operations to manage crops.

We chose this theme for several strategic reasons:

- **Relevance:** Agricultural automation is a burgeoning field with significant real-world impact.
- **Complexity Balance:** It offered a balanced mix of precision line following, complex decision-making algorithms, and mechanical manipulation.
- **Skill Development:** It provided an opportunity to master Reinforcement Learning (RL) alongside classical control systems like PID.

Initial Strategy: Our initial approach involved dividing the project into two parallel streams: Software Simulation (handled primarily using Python and CoppeliaSim) and Hardware Research (component selection and circuit design).

Stage 1 - Task 0: Environment Setup

2.1 Objective

Task 0 served as the foundational phase of the competition. The primary objective was to set up a robust development environment capable of handling complex simulations and to familiarize the team with the necessary software stack mandated by e-Yantra.

2.2 Software Installation

The following software tools and libraries were installed and configured:

- **Operating System:** Ubuntu 22.04 LTS was chosen for its stability and compatibility with ROS (Robot Operating System) and robotics development tools.
- **CoppeliaSim (formerly V-REP):** We installed CoppeliaSim Edu version. This powerful simulator was used to model the physics, sensors, and actuators of the Crop Drop Bot in a virtual environment.
- **Python 3.8+:** The primary programming language for logic implementation.
- **Required Libraries:**
- *NumPy:* For numerical computations and matrix operations.
- *OpenCV:* For image processing tasks involved in visual navigation.
- *PyRep/ZMQ:* For interfacing Python scripts with the CoppeliaSim environment.

2.3 Simulation Environment Configuration

Configuring the simulation environment involved establishing a communication bridge between our Python control scripts and the CoppeliaSim scene. We utilized the Remote API provided by CoppeliaSim.

Key Steps:

1. Loaded the provided `task0_scene.ttt` file into CoppeliaSim.
2. Verified the port connections for the Remote API.
3. Executed a "Hello World" script to confirm that Python could send commands (e.g. motor velocity) and receive data (e.g. sensor readings) from the simulator effectively.

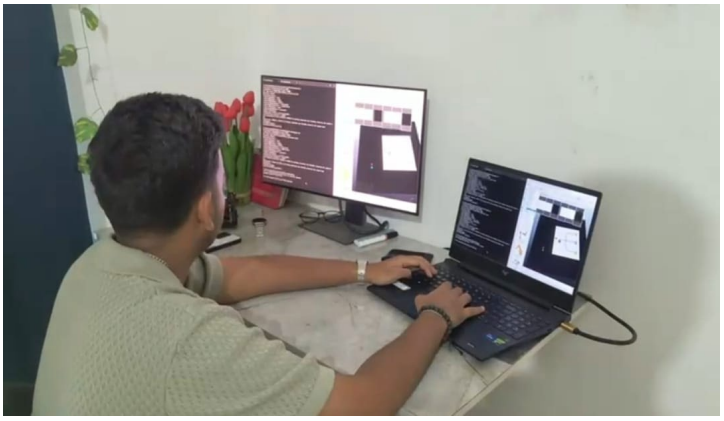
Outcome: By the end of Task 0, we had a fully functional simulation pipeline. We successfully verified that our codebase could control the virtual robot's motors and read data from its proximity sensors, laying the groundwork for the complex tasks ahead.

Stage 1 - Task 1: Basic Simulation & Control

3.1 Overview

Task 1 introduced the team to the dynamics of the robot. The goal was to move the robot from a start point to a goal point within the simulation without colliding with walls. This required the implementation of control algorithms.

3.2 PID Controller Implementation



To ensure smooth and accurate movement, we implemented a **Proportional-Integral-Derivative (PID)** controller. The PID controller continuously calculated an *error value* as the difference between a desired setpoint (e.g., center of the lane) and a measured process variable (e.g., sensor data).

- **Proportional (P):** Provided an immediate reaction to the current error.
- **Integral (I):** Accounted for past errors to eliminate steady-state error.
- **Derivative (D):** Predicted future errors based on the rate of change, providing damping to reduce overshoot.

3.3 Introduction to Reinforcement Learning (RL)

While PID was sufficient for line following, the competition required adaptive decision-making. We began experimenting with Q-Learning, a model-free Reinforcement Learning algorithm.

RL Setup in CoppeliaSim:

- **State Space:** Defined by the readings from the robot's vision sensors (line position relative to center).
- **Action Space:** Discrete actions including *Move Forward, Turn Left, Turn Right*.
- **Reward Function:** The agent received positive rewards for staying on the line and negative rewards for deviating or colliding.

3.4 Results

The combination of PID for low-level motor control and basic RL logic for navigation allowed the bot to traverse the Task 1 arena successfully. We tuned the Kp, Ki, and Kd constants extensively to minimize oscillation, resulting in a smooth trajectory.

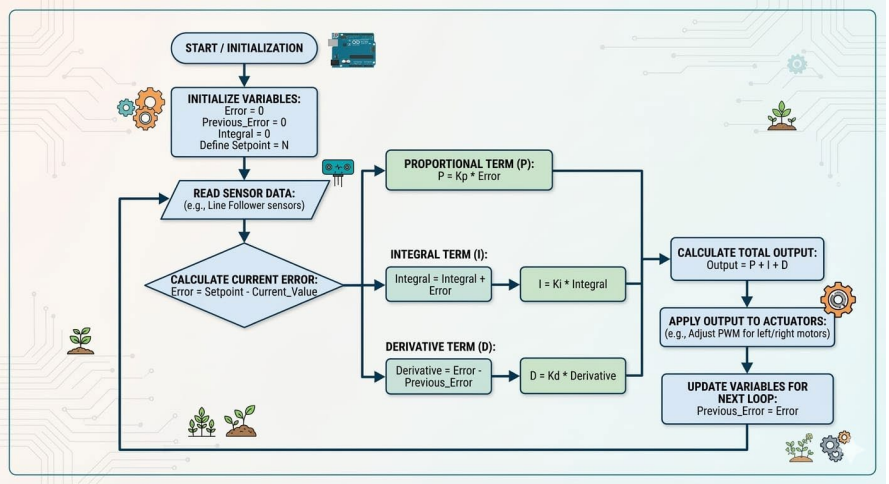
Stage 1 - Task 2: Complex Map Simulation

4.1 Challenge Description

Task 2 significantly increased the complexity of the simulation. The map now included sharp turns, intersections, and obstacles that mimicked the actual "Crop Drop" greenhouse environment. The robot had to navigate autonomously to specific coordinates.

4.2 Algorithm Modifications

The basic logic from Task 1 was insufficient for the complex geometry of Task 2. We upgraded our navigation stack to include:



- **Intersection Detection:** We utilized array-based processing of the vision sensor data to detect nodes (intersections). When the sensor array detected a horizontal line crossing the path, the robot identified it as a node.
- **Path Planning:** We implemented a graph-based search algorithm (Dijkstra's Algorithm) to determine the shortest path between the start node and the target crop zones.

4.3 Handling Constraints

The simulation imposed strict timing and collision constraints. To address this, we optimized our control loop:

- **Adaptive Velocity Profiling:** The robot was programmed to decelerate when approaching turns or intersections and accelerate during straight paths. This reduced the likelihood of overshooting the line during sharp maneuvers.
- **Sensor Noise Filtering:** We applied a moving average filter to the sensor data to smooth out noise in the simulation, ensuring stable control signals.

4.4 Challenges & Solutions

Challenge: The robot frequently lost the line at 90-degree turns.
Solution: We implemented a "recovery mode." If the sensors detected "all white" (off-line), the robot would rotate in the direction of the last known line position until the line was re-acquired.

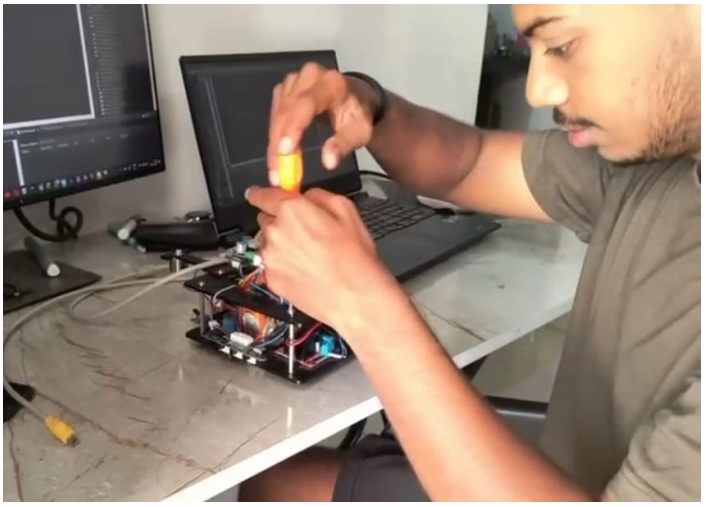
Stage 2 - Task 3: Hardware Assembly & Basic Kinematics

5.1 Transition to Hardware

Stage 2 marked the critical shift from virtual simulation to physical reality. Task 3 focused on assembling the "Crop Drop Bot" and validating its mechanical integrity.

5.2 Hardware Components

The kit provided by e-Yantra included:



- **Microcontroller:** STM32 F103RB
- **Actuators:** N20 Metal Gear Motors with encoders for precise odometry.
- **Sensors:** A custom-designed IR Sensor Array (5-channel) for line detection.
- **Power:** Li-ion Battery pack (11.1V).
- **Chassis:** Laser-cut acrylic chassis components.

5.3 Assembly Process

Mechanical Assembly: We assembled the chassis, mounting the motors and wheels in a differential drive configuration. Special attention was paid to the caster wheel to ensuring stability.

Electrical Connections: We wired the motor drivers (L298N) to the STM32 and connected the IR sensor array. We ensured proper cable management to prevent interference with moving parts.

5.4 Kinematics Testing

Before attempting line following, we wrote firmware to test basic kinematics:

1. **Straight Line Test:** Verified that both motors rotated at synchronized speeds to move the robot forward without drifting.
2. **Rotational Test:** Verified zero-radius turning (spinning in place) by driving motors in opposite directions.
3. **Encoder Validation:** We read encoder ticks to confirm that the physical distance moved matched the commanded distance.

Stage 2 - Task 4: Black Line Navigation

6.1 IR Sensor Calibration

The physical environment introduces variables not present in simulation, such as ambient light and uneven surface friction. The first step in Task 4 was robust sensor calibration.

We implemented a calibration routine where the robot rotated over the line, recording the minimum (black) and maximum (white) analog values for each of the 5 sensors. These values were used to normalize readings to a 0-1000 range, ensuring consistent performance regardless of lighting changes.

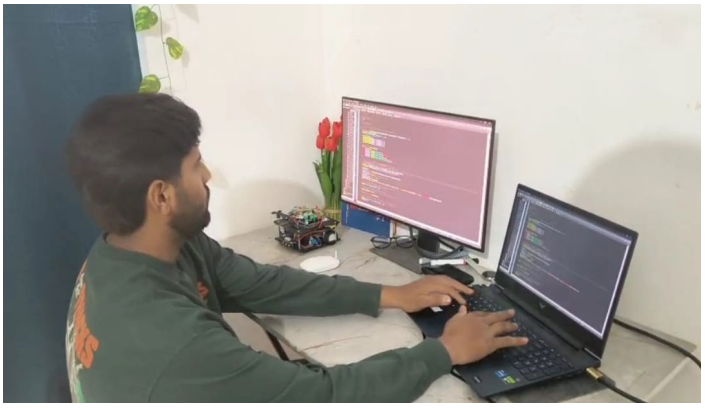
6.2 Line Following Logic

We utilized a weighted average algorithm to determine the line's position relative to the robot's center:

```
Position = Σ(Sensor_Value * Weight) / Σ(Sensor_Value)
```

This calculated position was fed into a hardware-tuned PID controller.

6.3 PID Tuning for Hardware



Tuning PID on hardware proved more challenging than simulation due to physical inertia and battery voltage fluctuations.

- **Kp (Proportional):** Increased until the robot followed the line but oscillated rapidly.
- **Kd (Derivative):** Added to dampen the oscillations and smooth the movement. This was critical for high-speed tracking.
- **Ki (Integral):** Kept very low to address minor drifts without causing instability.

6.4 Precision Movement

To handle the "stops" required at crop zones, we integrated encoder feedback. The robot could now execute commands such as "Follow line for 30cm, then stop," bridging the gap between line following and odometry-based navigation.

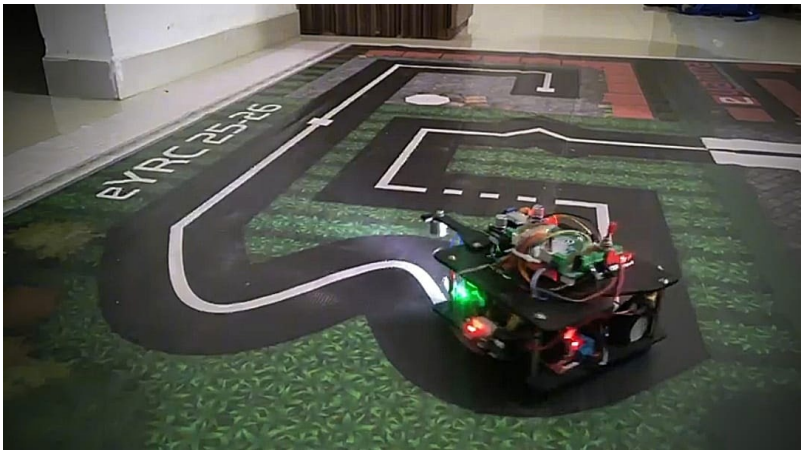
Stage 2 - Task 5: Advanced Navigation & Manipulation

7.1 Dual-Mode Navigation (Black vs. White Lines)

Task 5 introduced a complex arena featuring both black lines on a white background and white lines on a black background. The robot had to dynamically switch its interpretation of sensor data.

Inversion Logic: Implemented a state variable `line_mode`. When the robot detected a specific marker (a barcode-like pattern on the floor), it inverted the logic of the sensor array (treating high signals as low and vice-versa) to seamlessly transition between zones.

7.2 Intersection Decision Making



The arena was a grid of intersections. We developed a node-counting algorithm:

- The robot counted intersections (nodes) as it traversed.
- Based on a pre-loaded map array, the robot decided whether to go straight, turn left, or turn right at each specific node count.
- This determinist approach ensured the robot followed the exact required path to the crop zones.

7.3 Manipulation Mechanism

We designed and 3D-printed a gripper mechanism powered by a servo motor to pick up the "crop" blocks.

- **Design:** A simple electromagnetic gripper.
- **Integration:** The gripper was mounted at the side of the chassis. The code included specific functions `pick_crop()` and `place_crop()` which adjusted the magnetic clampling action .
- **Alignment:** Accurate stopping was crucial. We fine-tuned the braking logic to ensure the gripper was perfectly aligned with the crop block when the robot stopped.

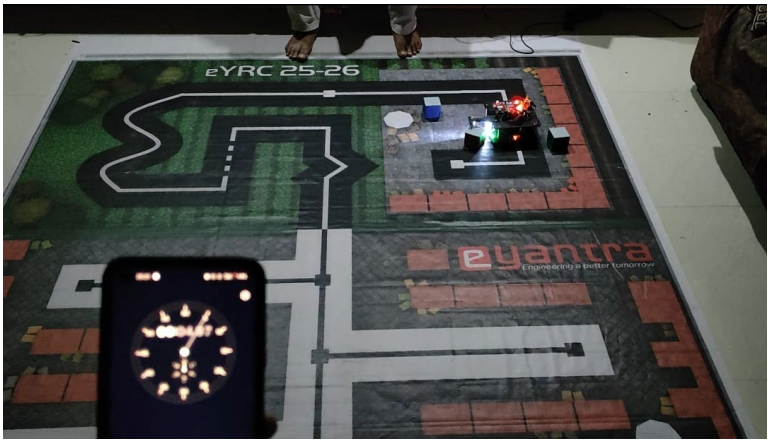
Stage 2 - Task 6: The Semifinal Run

8.1 The Semifinal Challenge

Task 6 was the culmination of our efforts. It required a complete, autonomous run on the full physical arena. The robot had to: 1. Start from the depot. 2. Navigate to multiple crop zones. 3. Pick up crops. 4. Transport them to specific drop zones based on color codes. 5. Return to the start point.

8.2 Integrated Run Execution

The final code merged the navigation stack, the manipulation logic, and the path planning algorithms into a single robust state machine.



Performance:
During the semifinal video submission, our Crop Drop Bot successfully navigated the arena. It demonstrated precise line switching (Black to White) and accurate intersection turning. The gripper mechanism successfully picked up the designated crops.

8.3 Challenges Faced

Despite our success, we faced significant challenges:

- **Lighting Sensitivity:** The semifinal arena had varying lighting conditions which initially caused false triggers on the sensors. We resolved this by implementing dynamic thresholding in the software.
- **Power Drain:** The voltage drop during motor operation occasionally reset the microcontroller. We added a large capacitor across the power rails to buffer these spikes.

8.4 Result

We completed the run with high accuracy. While we did not advance to the finals based on timing metrics, our technical implementation was highly rated, leading to further opportunities with IIT Bombay.

The Online Interview, IIT Bombay Visit & Conclusion

9.1 Internship Interview

Based on our strong performance in the semifinals, our team was shortlisted for an internship interview with the e-Yantra labs at IIT Bombay.

The interview was a rigorous technical assessment conducted online by IIT Bombay mentors. We defended our code logic, explained our choice of PID constants, and discussed the intricacies of our sensor calibration routine. The panel tested our understanding of:

- Embedded C programming nuances.
- Algorithm efficiency (Big O notation of our path planning).
- Electrical circuit isolation and motor driver characteristics.

(Results of this interview are currently pending.)

9.2 Invitation to IIT Bombay

We have received an official invitation to visit the IIT Bombay campus as guests during the finals. This visit serves three purposes:

1. **Observation:** To witness the Grand Finale and learn from the top competing teams.
2. **Hardware Return:** To officially return the robotic components provided for the competition.
3. **Certification:** To collect our certificates of completion and merit in person.

9.3 Skills Acquired

Technical Skills:
We have gained proficiency in **CoppeliaSim** for simulation, **Python & Embedded C** for control logic, and **PID Control** systems. We also gained practical experience in **Reinforcement Learning** basics and hardware debugging.

Soft Skills:
The journey honed our teamwork, crisis management (debugging hardware hours before deadlines), and technical documentation skills.

9.4 Conclusion

Participating in eYRC 2025-26 has been a transformative experience for the team from Zeal College of Engineering and Research. While we stopped at the semifinals, the knowledge gained and the recognition received from IIT Bombay have set a strong foundation for our future careers in robotics and automation.